

Practice Lab:

Edit and Compile using Eclipse with Scala

- This Practice Lab has as a prerequisite, Practice Lab 6241, where we install and configure Eclipse Photon on MacOS including a Scala plugin.
- This Practice Lab also has a prerequisite, Discussion Units 7544/7545 and 7548/7549, where we make the DSE Analytics client we run here.
- In this Practice Lab, we edit, compile, and run a DSE Analytics client program. As such, this lab expects an operating DSE cluster with DSE Analytics be accessible.
- Instructions in this practice lab are written for MacOS, but should function for most Linux distros.
- This Practice Lab will require reasonable Internet access.

Eclipse IDE: What is it ?

Open source integrated developer's workbench (IDE)

- From 1984's Visual Age, a similar IBM program
- Not apache, is Eclipse Public License (ECL)
- Releases with science related names
- June/2018 release is Photon
- Many/many development languages supported
- Extensible (Vi plugin !)



Eclipse: How to Install

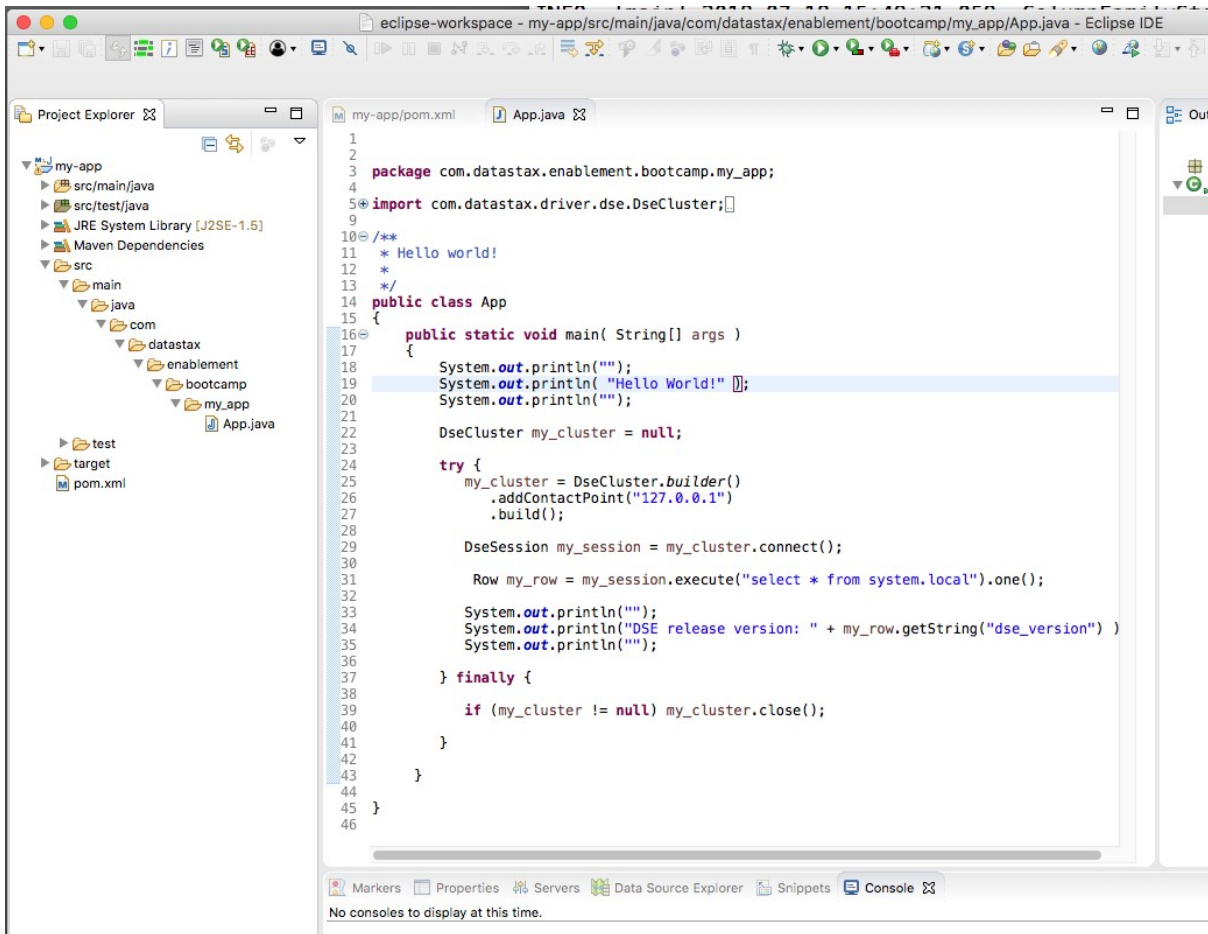


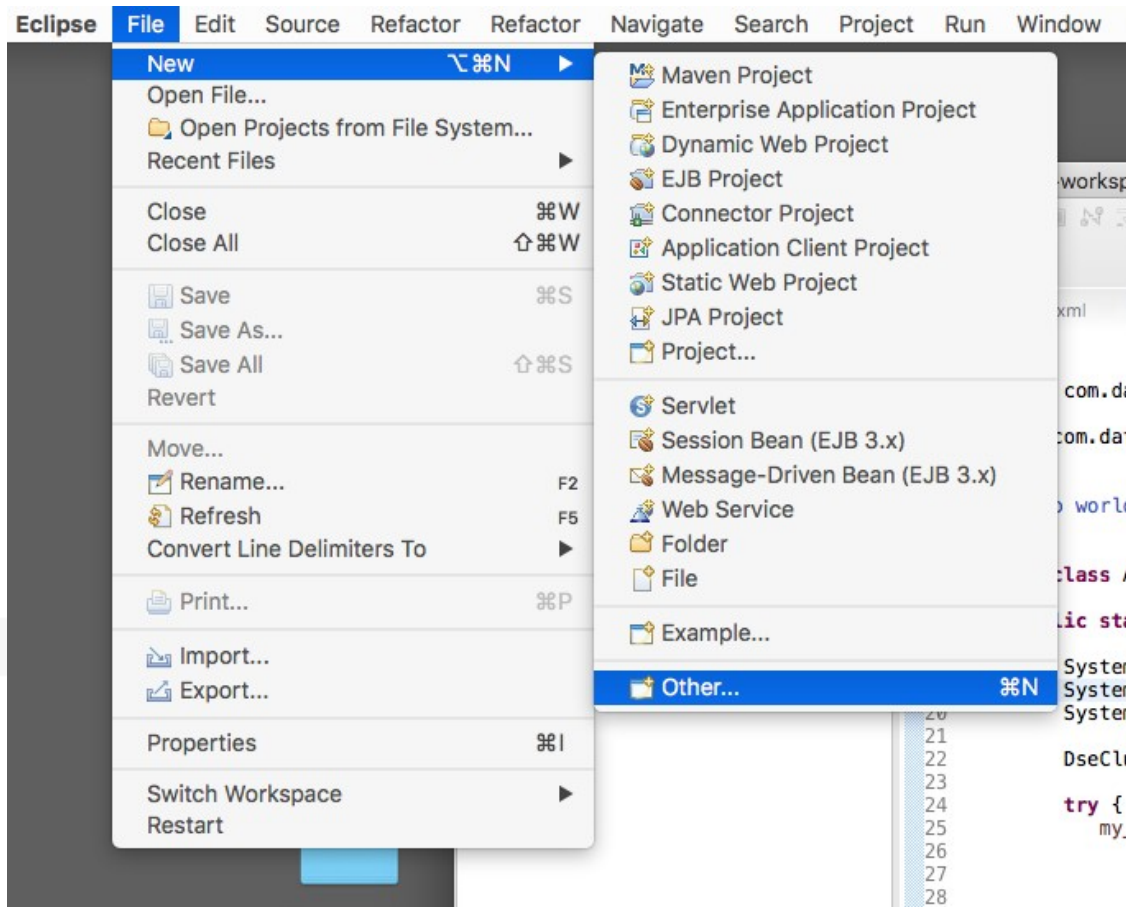
**Instructions in Practice
Lab 6241.**

- Reference Practice Lab 6241, where we installed Eclipse, and an Eclipse plugin specific to Scala.
- Launch Eclipse, and then ..

After Practice Lab 6241-

- Configured for DSE Java clients
- Eclipse Scala plugin installed



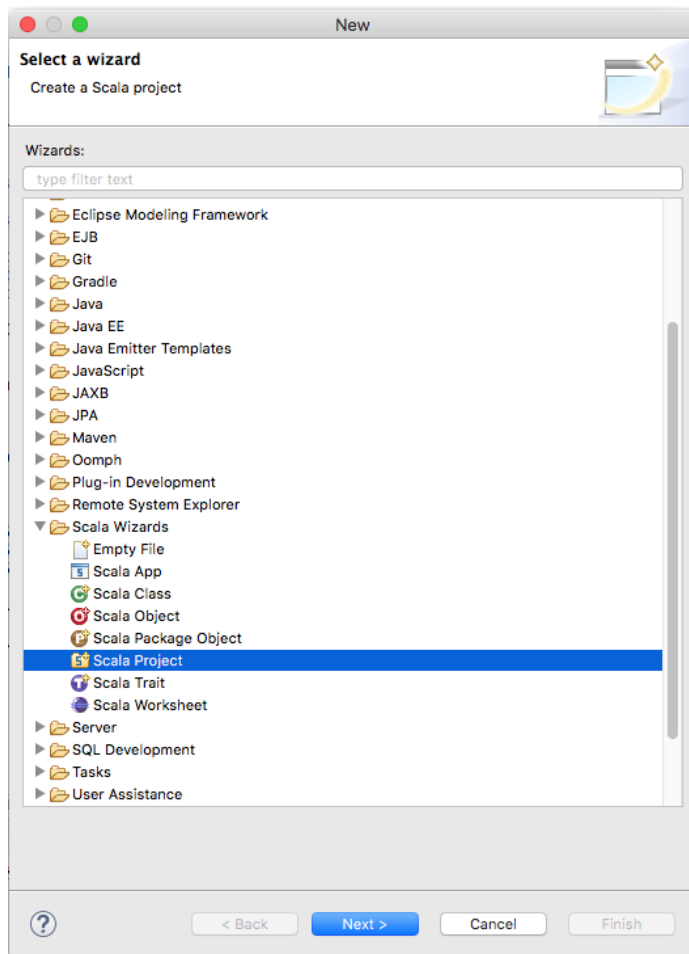


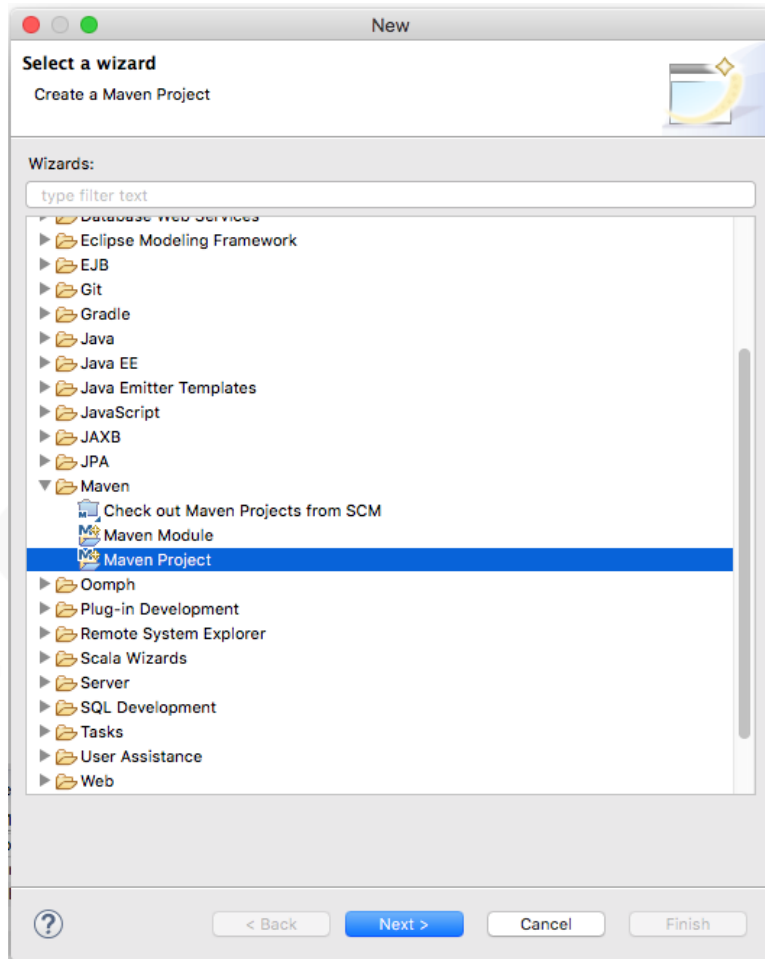
From the Eclipse Menu Bar-

- File -> New -> Other

Don't use this-

- Simple Scala programs only
- Not a Maven project, no pom.xml
- Better off using the DSE Spark REPL



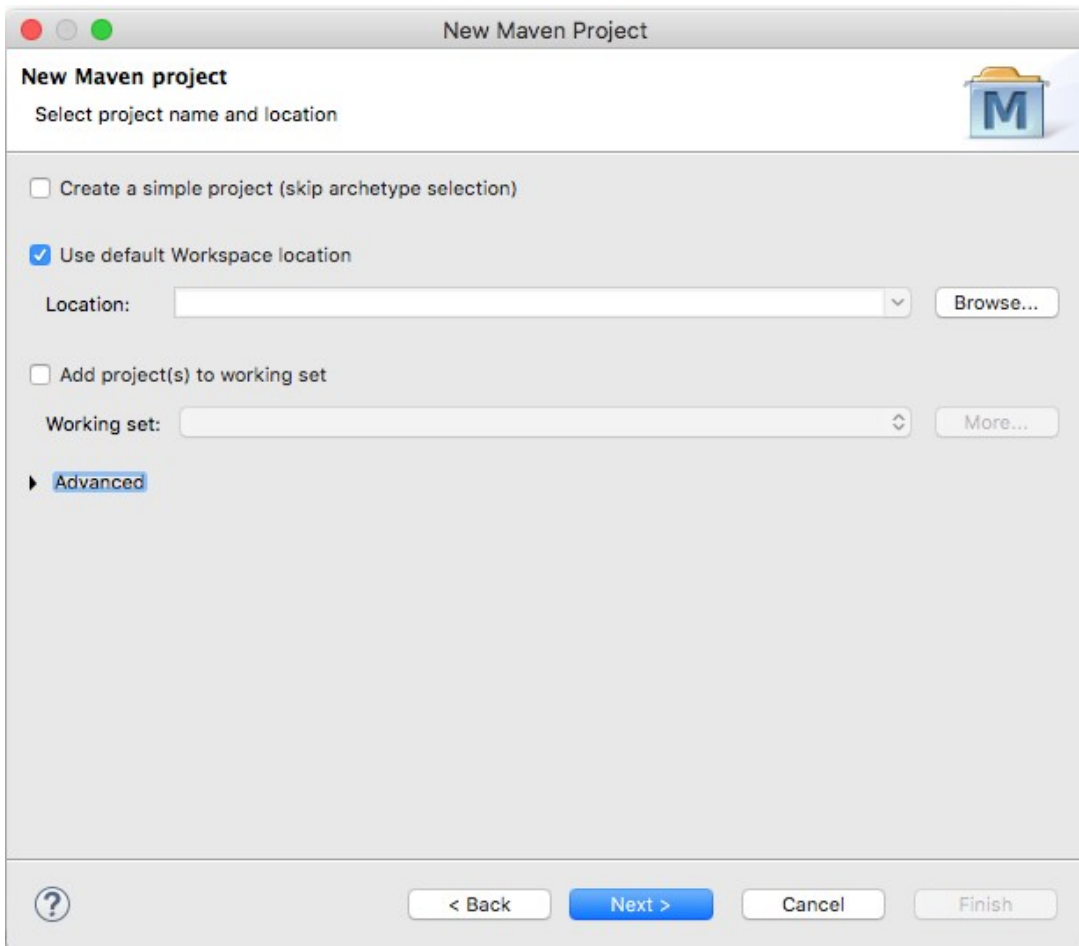


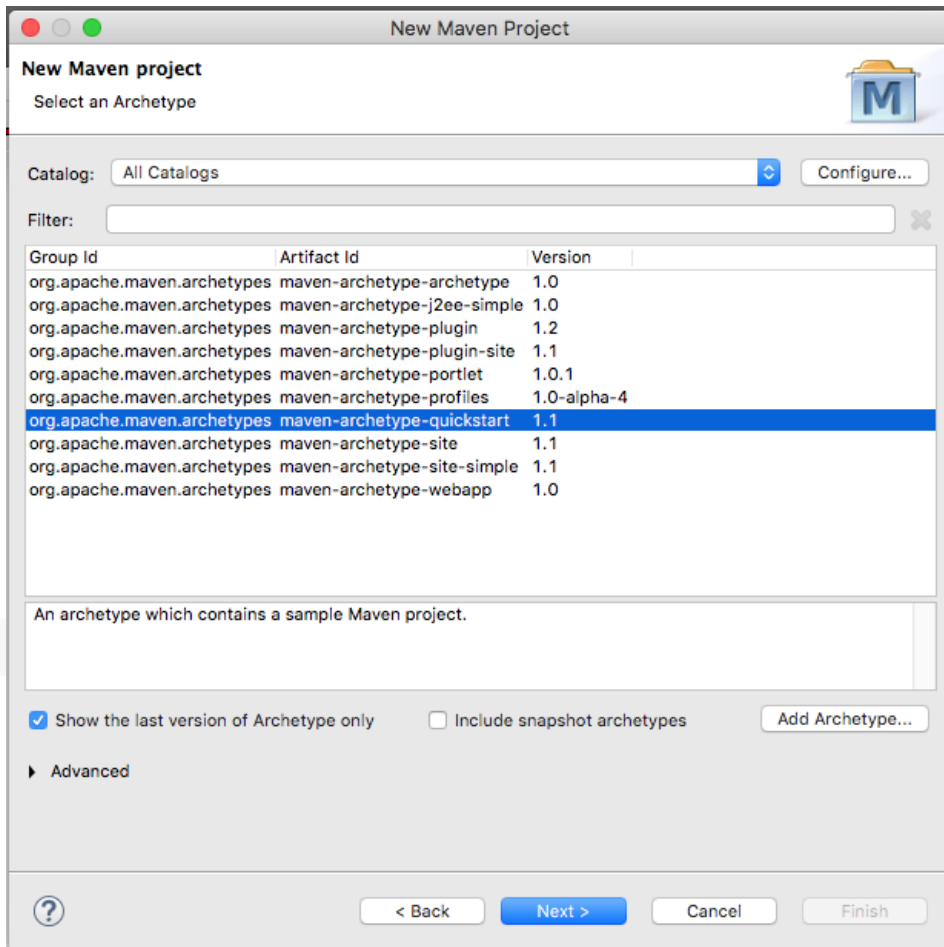
In the, Select a wizard, modal dialog box

- Maven -> Maven Project
- This option gives us support for a pom.xml, which will automatically pull all of our (DSE and Spark) dependencies

Just click through this dialog box-

- There's nothing we need on this dialog box-
- Click, Next



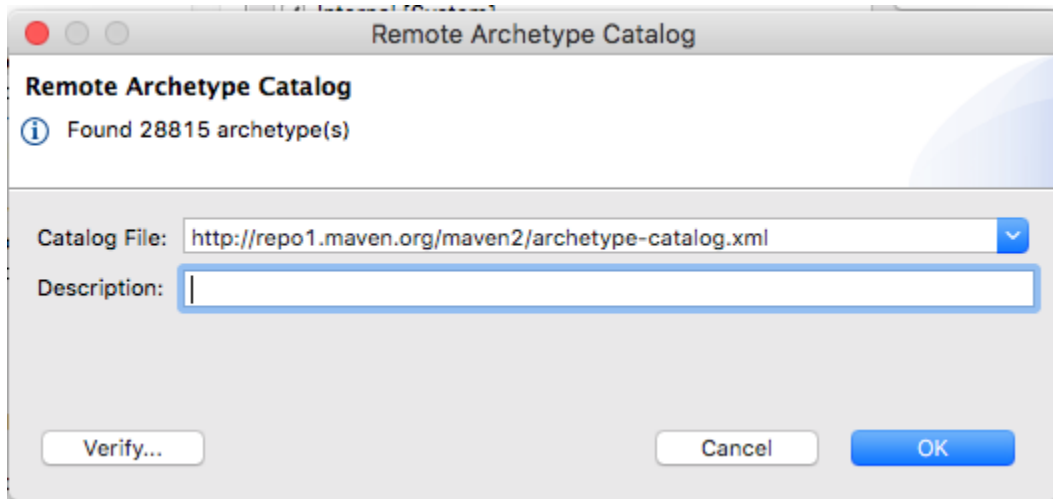


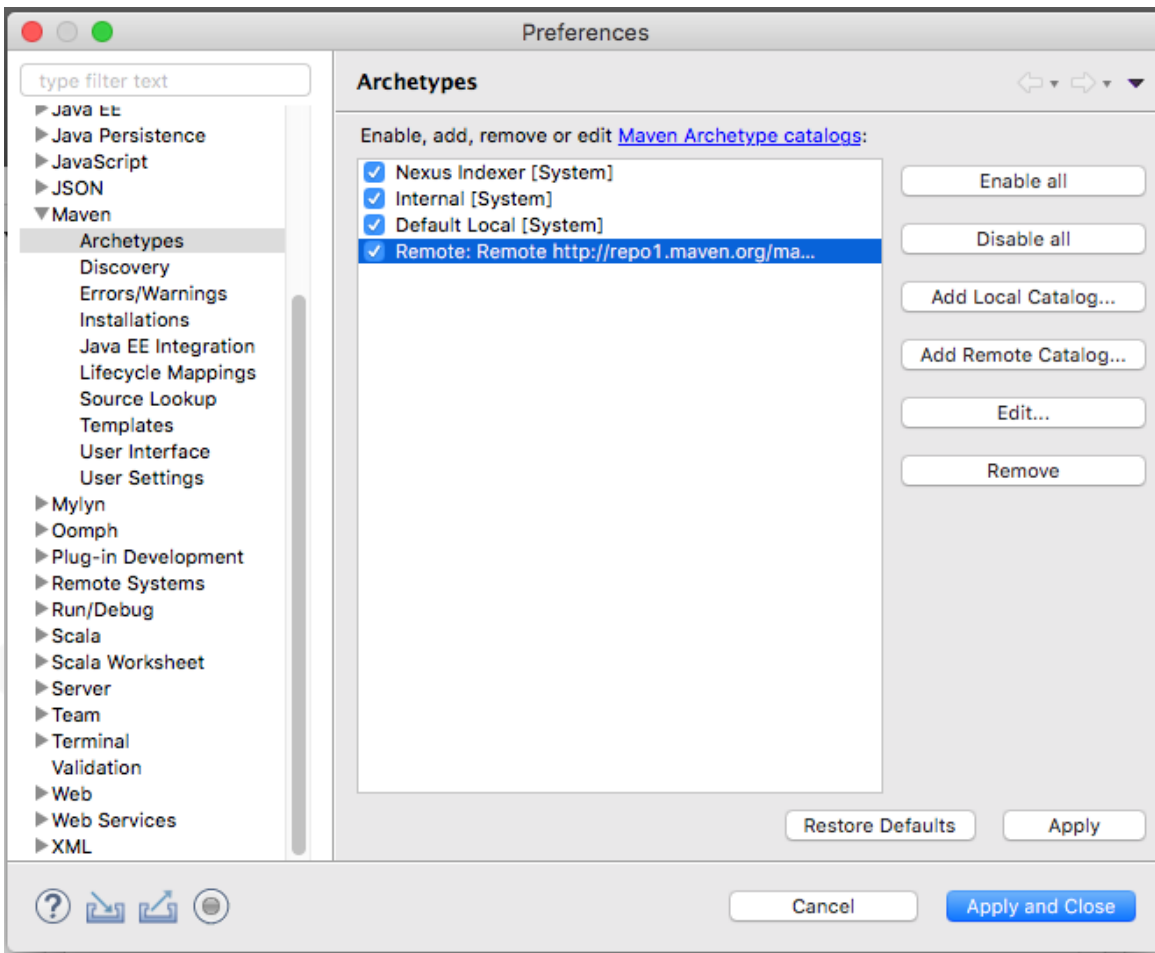
Maven Archetypes

- Maven Archetypes, a project templating device; filesystem structure, other
- The default is the same we used in Discussion Unit 6240, the default Java client archetype
- We want Scala
- Click, Configure

Online (Remote) Maven Archetype Repository

- Enter the value as shown in the text entry field titled, Catalog File
- Optionally Click, Verify
- And Click, OK



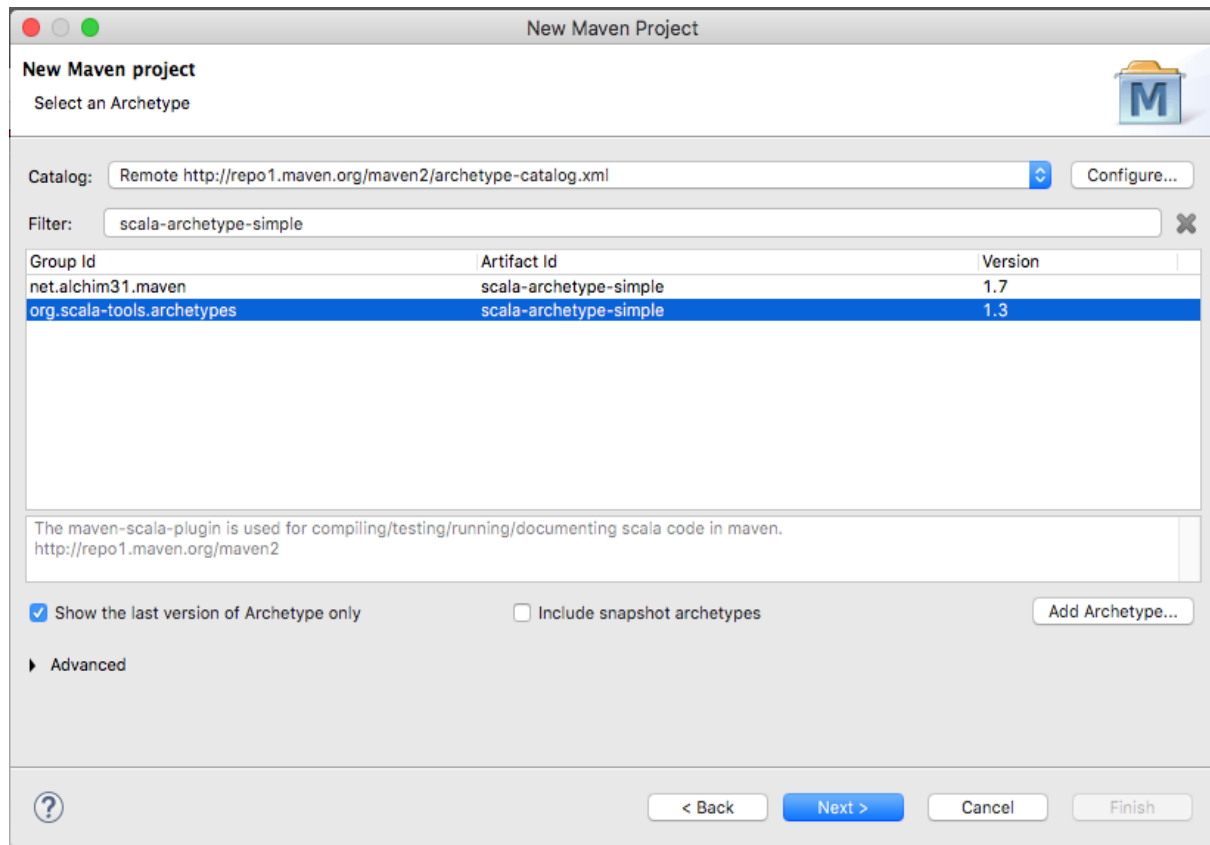


Archetype Catalog added-

- After the addition of our Archetype Catalog, Click, Apply and Close

Picking a given Archetype

- Manage the display as shown on the right-
- Specifically enter a Filter of, scala-archetype-simple, to reduce the number of options
- When your dialog box matches that as shown, Click, Next



The Project Proper-

- Enter a GroupId, ArtifactId, and Version as shown
- Click, Finish

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

Properties available from archetype:

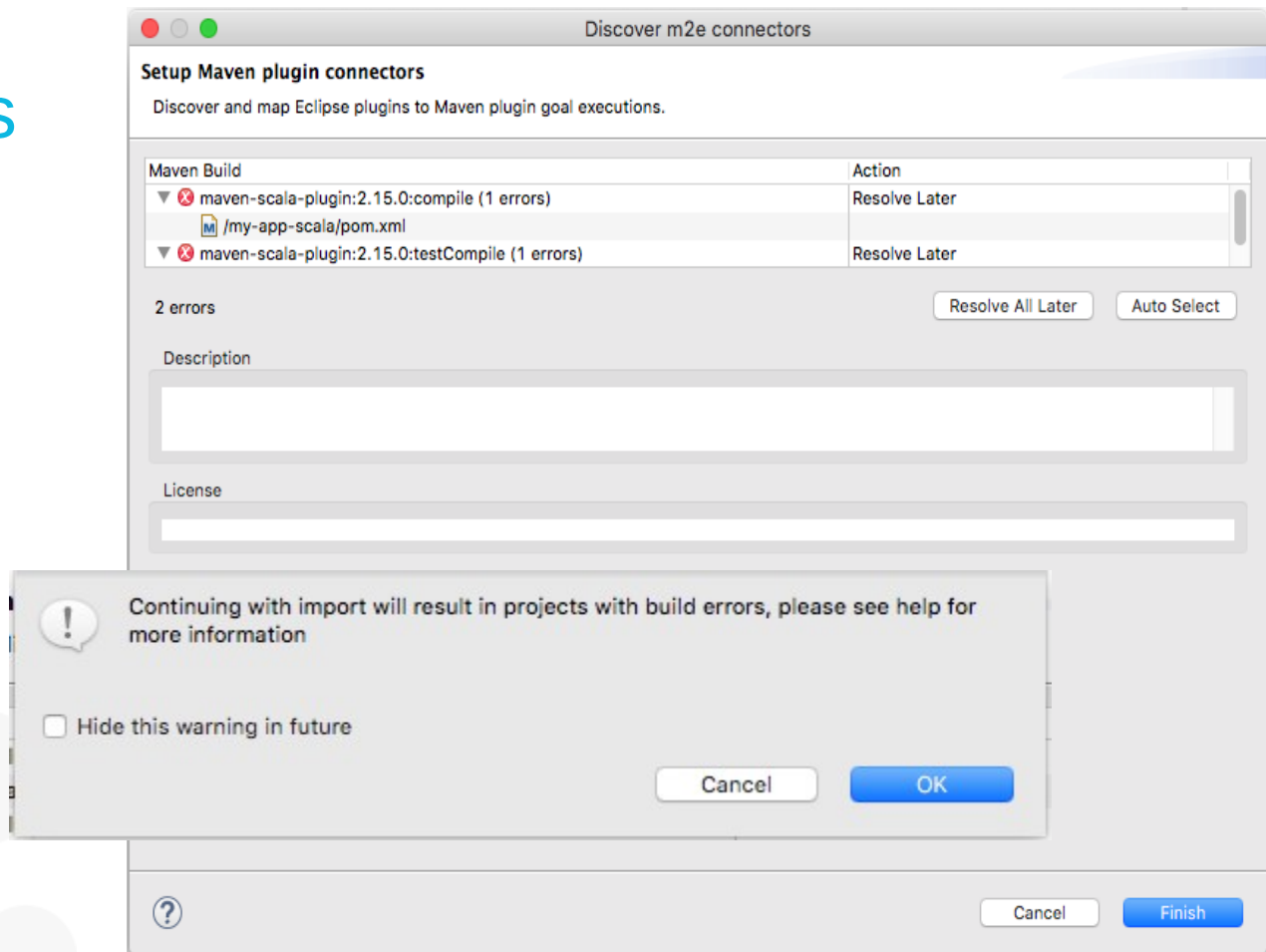
Name	Value

Advanced

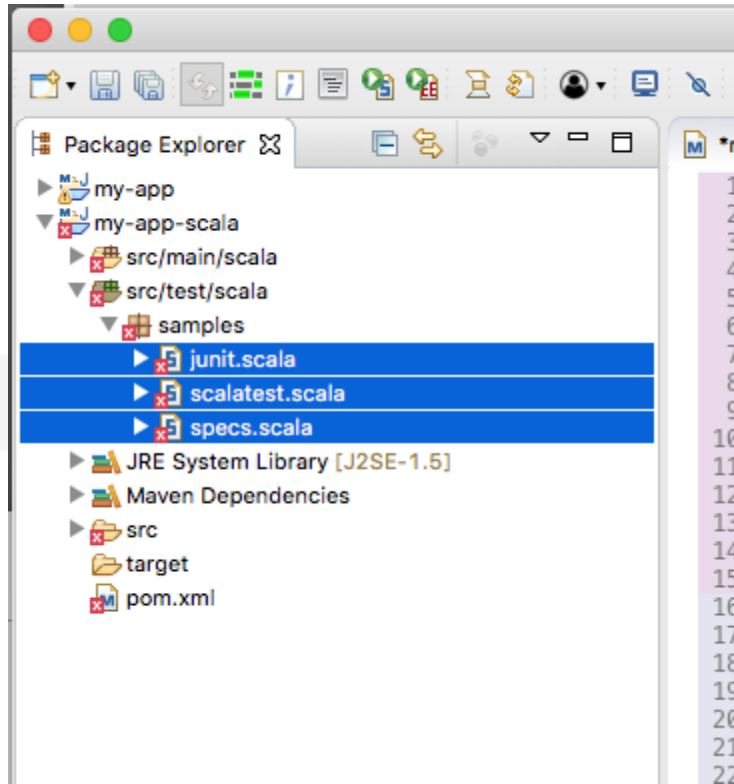
< Back Next > Cancel Finish

As generated, a number of things are broken-

- We'll fix these; not here

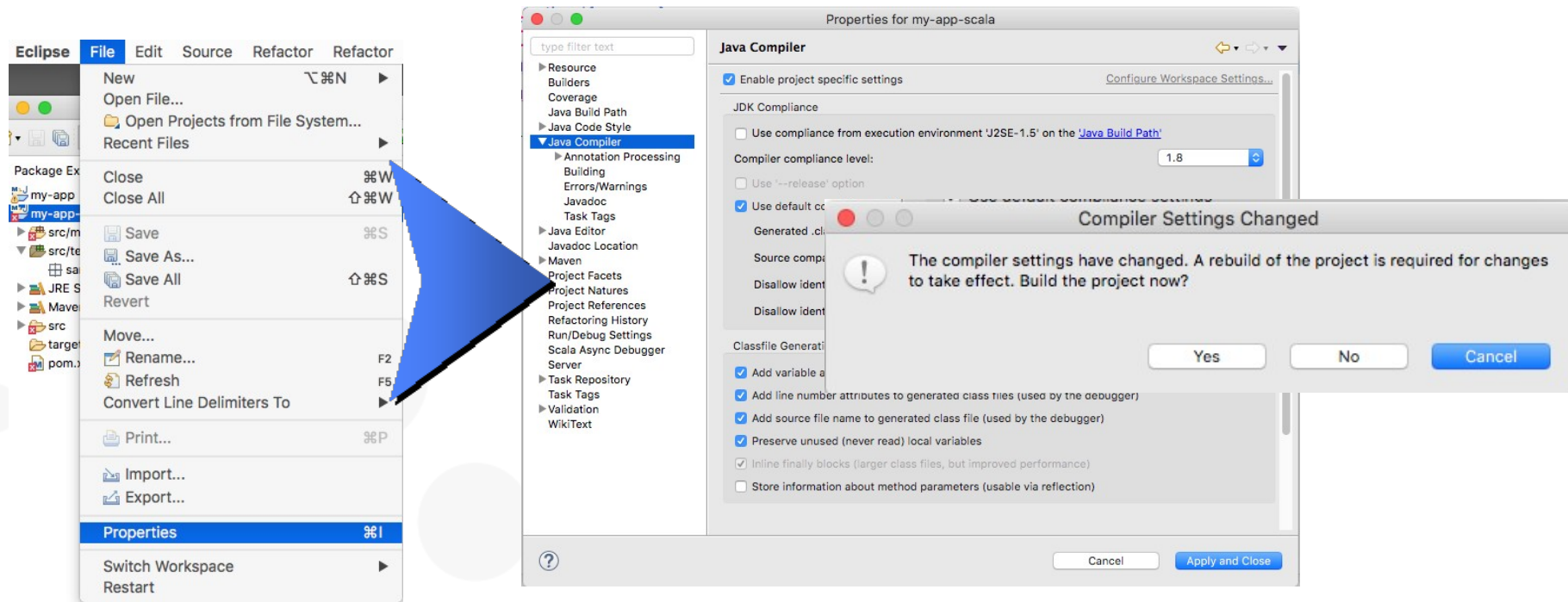


Delete the generated/stub, test code



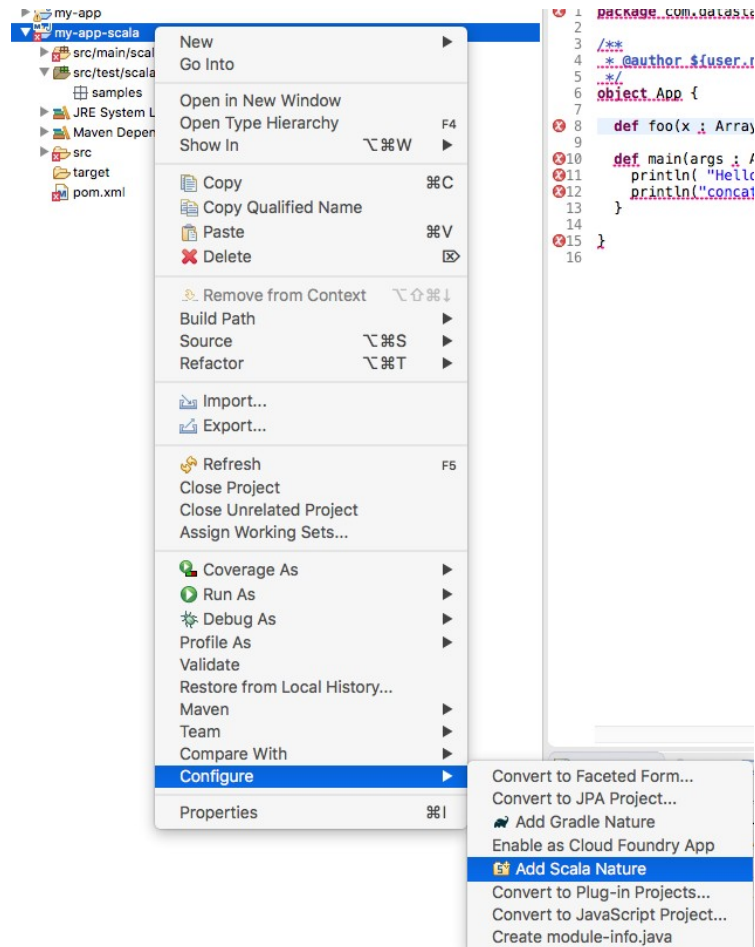
- In the Project Explorer view, navigate to, my-app/scala -> src/test/scala -> samples
- Shift-Click all entries, Right-Click, and Delete
- These are stub test units and do not compile

Setting Java Compiler version to 1.8-



Maven knows, Eclipse doesn't know

- While Maven knows we want a Scala project, Eclipse does not
- In the Project Explorer view, Right-Click the project, and select; Configure -> Add Scala Nature



```

my-app-scala/pom.xml  App.scala
65     </dependency>
66 </dependencies>
67
68 <repositories>
69   <repository>
70     <id>DataStax-Repo</id>
71     <url>https://repo.datastax.com/public-repos</url>
72   </repository>
73 </repositories>
74
75 <build>
76   <plugins>
77     <!--
78     <plugin>
79       <groupId>net.alchim31.maven</groupId>
80       <artifactId>scala-maven-plugin</artifactId>
81       <version>3.2.2</version>
82       <executions>
83         <execution>
84           <phase>process-sources</phase>
85           <goals>
86             <goal>compile</goal>
87             <goal>testCompile</goal>
88           </goals>
89           <configuration>
90             <mainSourceDir>${project.build.sourceDirectory}/../scala</mainSourceDir>
91           </configuration>
92         </execution>
93       </executions>
94     </plugin>
95     <!--
96     <plugin>
97       <groupId>org.apache.maven.plugins</groupId>
98       <artifactId>maven-shade-plugin</artifactId>

```

Fix the pom.xml-

- Edit the pom.xml file as detailed on the Notes page.
- Save
- From the Menu Bar; Project -> Clean
- (Clean can take a bit of time.)

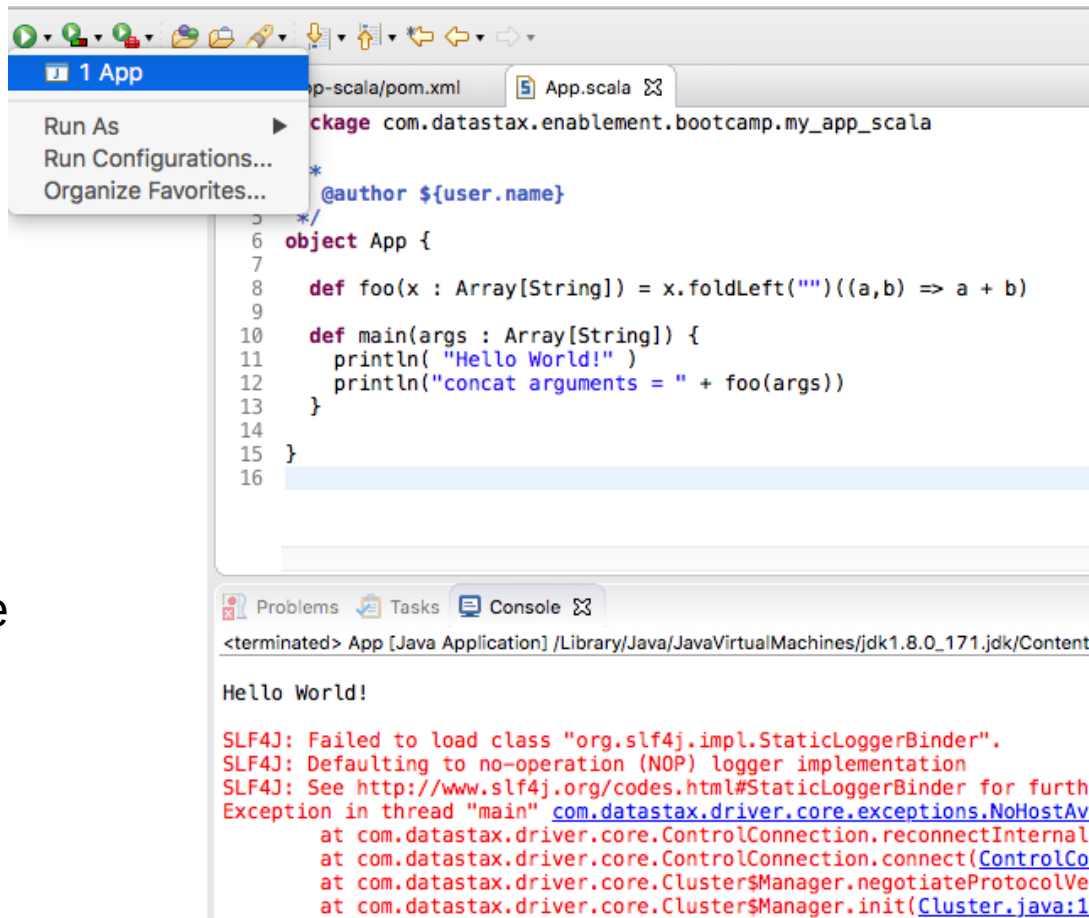
What is a pom.xml ?

- xml
- "project object model"
- Each project has one
- Tells Maven how to compile, etc, your project
- Edit once and done ?

```
<project xmlns="http://maven.apache.or ...  
  xsi:schemaLocation="http://mav ...  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>com.datastax.enablement.bootcamp<  
/groupId>  
  <artifactId>my-app</artifactId>  
  <packaging>jar</packaging>  
  <version>1.0-SNAPSHOT</version>  
  <name>my-app</name>  
  <url>http://maven.apache.org</url>  
  <dependencies>  
    <dependency>  
      <groupId>junit</groupId>  
      <artifactId>junit</artifactId>  
      <version>3.8.1</version>  
      <scope>test</scope>  
    </dependency>  
  </dependencies>  
</project>
```

Run the generated App.scala

- Time for a test-
- In the Project Explorer view, find and open (Double-Click) App.scala
- From the Eclipse Toolbar, click the Run icon (Run is context sensitive, and App.scala must be current.)
- Successful output as shown in the Console view



The screenshot shows the Eclipse IDE interface. The top toolbar has the Run icon (a green play button) highlighted. Below it, a context menu is open with options: Run As, Run Configurations..., and Organize Favorites... The main editor displays the App.scala file with the following code:

```
1 package com.datastax.enablement.bootcamp.my_app_scala
2
3 @author ${user.name}
4
5 /**
6  *
7  */
8 object App {
9
10     def foo(x : Array[String]) = x.foldLeft("")((a,b) => a + b)
11
12     def main(args : Array[String]) {
13         println( "Hello World!" )
14         println("concat arguments = " + foo(args))
15     }
16 }
```

The bottom console view shows the output of the application:

```
<terminated> App [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/Content
Hello World!
```

Below the console output, there is a red error message:

```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for furth
Exception in thread "main" com.datastax.driver.core.exceptions.NoHostAv
    at com.datastax.driver.core.ControlConnection.reconnectInternal
    at com.datastax.driver.core.ControlConnection.connect(ControlCo
    at com.datastax.driver.core.Cluster$Manager.negotiateProtocolVe
    at com.datastax.driver.core.Cluster$Manager.init(Cluster.java:1
```

That was the default generated App.scala-

```
: All host(s) tried for query failed (tried: /127.0.0.1:9042 (com.datastax.driver.core.exceptions  
UnreachableException: java:237)
```

- From Discussion Unit 7548/7549, and 7554, grab the very same pom.xml, and App.scala you created and ran there.
- Copy and paste the above files into the same pom.xml and App.scala you just generated and ran. Be certain to Save.
- Do a, Project -> Clean, and Run

```

99 // -----
100
101
102 val My_Schema = StructType(Array(
103     StructField("customer_num"    , IntegerType, true),
104     StructField("fname"           , StringType, true),
105     StructField("lname"           , StringType, true),
106     StructField("company"         , StringType, true),
107     StructField("address1"        , StringType, true),
108     StructField("address2"        , StringType, true),
109     StructField("city"            , StringType, true),
110     StructField("state"           , StringType, true),
111     StructField("zipcode"         , StringType, true),
112     StructField("phone"           , StringType, true)

```

You are done when-

Problems Tasks Console

<terminated> App [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_171.jdk/Contents/Home/bin/java (Jul 18, 20

Hello World!

SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
 SLF4J: Defaulting to no-operation (NOP) logger implementation
 SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.

DSE release version: 6.0.0



Practice Lab:

Lessons Learned





End of Unit:

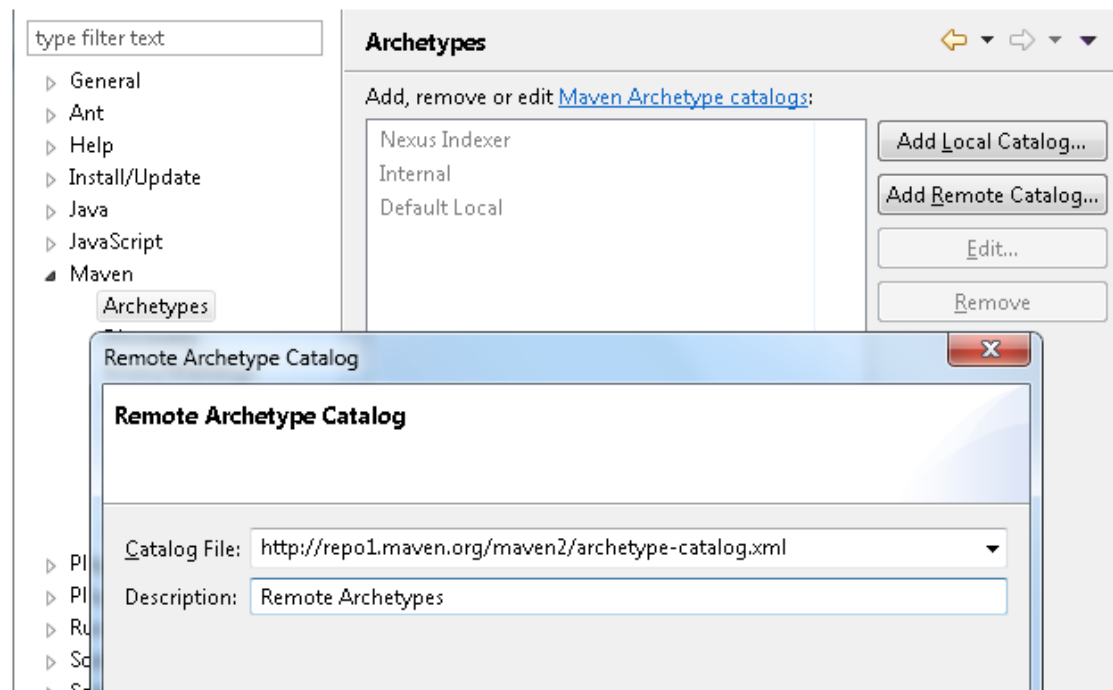


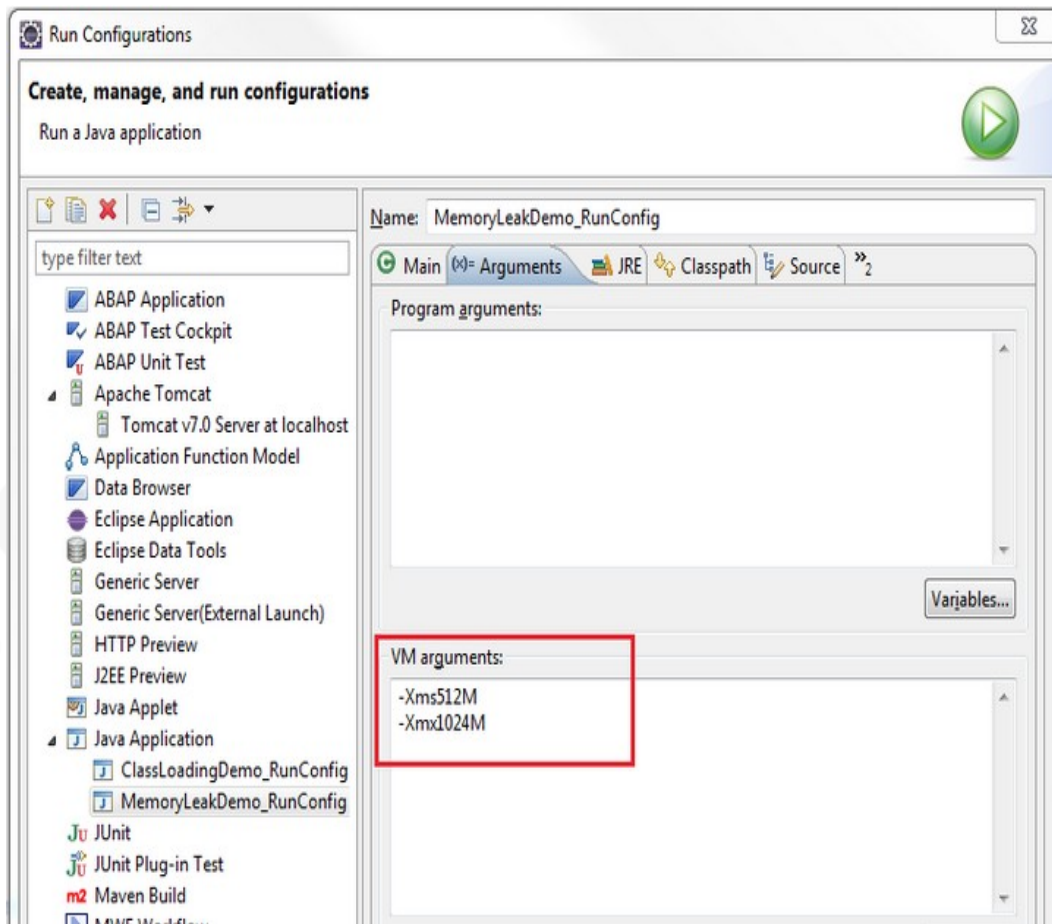
Additional Detail:

More Help Finding Archetypes:

The Notes page lists a Url to a StackOverflow article on finding Maven Scala archetypes-

Go to [Windows] → [Preferences] → [Maven] → [Archetypes] → **Add Remote Catalog** and create the catalog, then **Apply**





Eclipse Sluggish ?

- Eclipse is a Java app
- Tune the JVM settings as shown; specifically Xmx
- 1G default is common, 4G if you have room
- [Url on Notes page](#)